

ARQUITETURAS DE COMPUTADORES

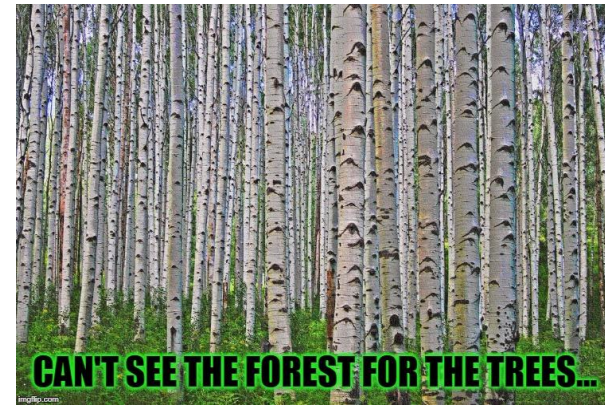
TURMA A1

TÓPICO 2 – MICROARQUITETURA (AULA 16)

Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

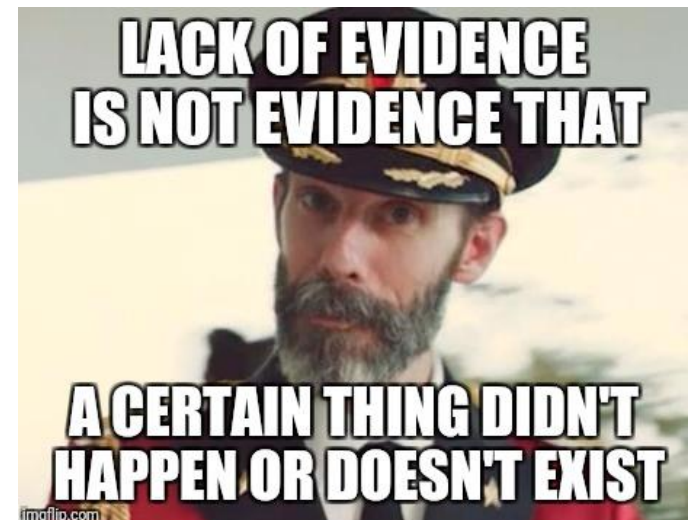
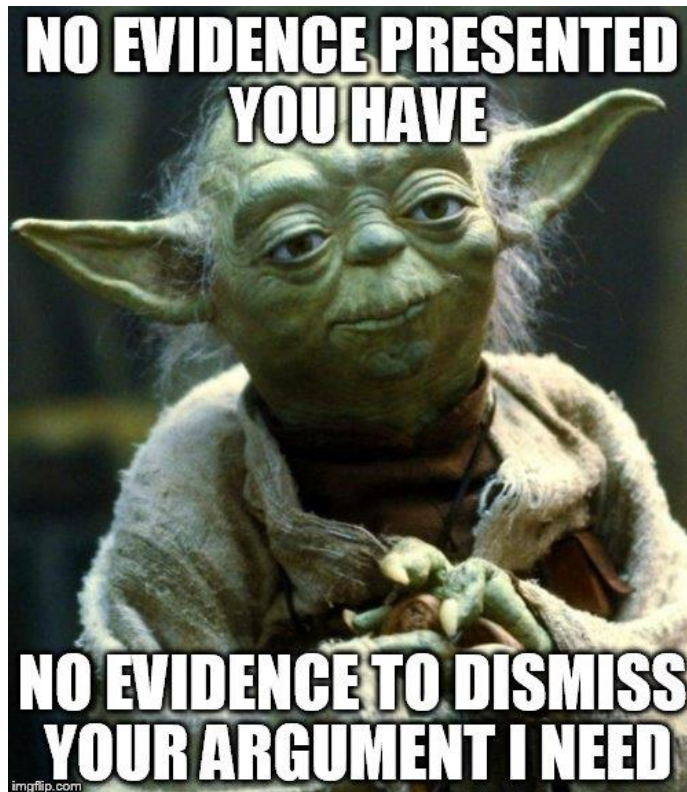
Avaliação de Tópico 1

- Considere que uma instrução do nível n pode fazer o trabalho de b instruções do nível $n-1$, ou seja, uma instrução no nível n é b vezes mais poderosa do que uma instrução no nível $n-1$. Se um programa de nível 1 requer s segundos para ser executado, quanto tempo programas equivalentes levariam nos níveis 2 e 3, assumindo que p instruções no nível n são necessárias para interpretar uma única instrução no nível $n+1$?
- É uma pergunta simples, mas exemplifica a dificuldade que muitos tem
 - de ler e entender o que está sendo solicitado e
 - de não ver a floresta por causa das árvores!



Macro- x Micro-programação

- Um compilador deve gerar uma saída para nível 1 ao invés de nível 2?
 - Na prática, seria inviável devido às complexidades das linguagens e arquiteturas e por ser uma solução ineficiente, cara, lento e de difícil implementação!



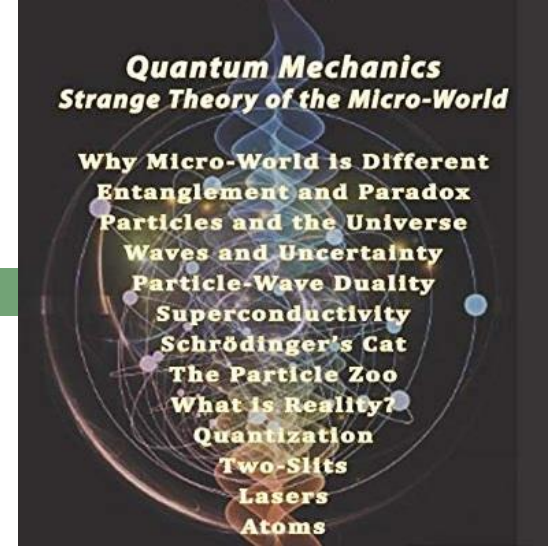
Macro- x Micro-programação

- Um compilador deve gerar uma saída para nível 1 ao invés de nível 2?
 - Na prática, seria inviável devido às complexidades das linguagens e arquiteturas e por ser uma solução ineficiente, cara, lento e de difícil implementação!
 - Enquanto sem a interpretação, a execução direto de μ insts poderia **aparecer de ser mais rápido**. Na verdade ...
 - Vai precisar de mais memória para guardar o programa: um MC grande é muito caro; além disso, deve ser **acessível** e possível **escrever** nela; [Custo ✗]
 - Se usa MP, por ser mais lento, a execução fica mais lento [Desempenho ✗]
 - Compilador precisa explorar a μ arq., μ insts, os registradores, concorrência, etc.
 - Número de μ insts é bem maior do que macroinsts, mais complexidade [Facilidade ✗]
 - Cada nova arquitetura tem novas μ insts $\rightarrow$ um novo compilador, e mais custos
 - Lembre-se da quantidade compiladores que serão necessários: $m \times n$ em vez de m

Microarquitetura é quantum?

❑ Será que a microarquitetura é a mecânica quântica do mundo da computação?

- O projeto específico de circuitos, pipelines e caches que implementa uma Arquitetura de Conjunto de Instruções (ISA) serve como a camada fundamental e "oculta" da computação, onde o desempenho teórico se torna realidade física.
- Assim como a mecânica quântica explica os comportamentos da matéria, muitas vezes contraintuitivos, complexos e altamente sensíveis, a microarquitetura gerencia as interações incrivelmente complexas, rápidas e frequentemente imprevisíveis dos processadores modernos.
- Microarquitetura - tornando o impossível possível



Microprogramação

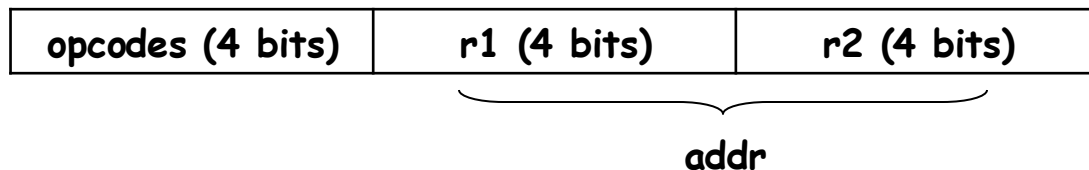


- **Microprogramação Horizontal x Microprogramação Vertical (MV)**
 - MIC-1 é um exemplo de microprogramação horizontal
 - O opcode é composto de sinais de controle, explicitamente, então as μ insts estão largas, com muitos bits
 - Assim, suporta a realização de várias **micro-operações** na mesma μ instrução
 - Por ex., `mar := ac; mbr := lshift(mbr + ac); wr; if n goto 23`
 - Podemos projetar um MIC-2 que emprega MV com a mesma VD
 - Por exemplo, suas μ instruções terão só 12 bits, com opcodes de só 4 bits
 - Quantos μ insts na μ linguagem de MIC-2 x MIC-1?
 - O microprograma porém teria 161 μ instruções, por ter instruções mais simples
 - Mas o microprograma do MIC-2 só ocupa 1920 bits, 24% a menos dos 2528 bits do microprograma do MIC-1!

As Microinstruções de MIC-2

Binário	Mnemônico	Função	Significado
0000	ADD	Adição	$r1 := r1 + r2$
0001	AND	E booleano	$r1 := r1 \text{ AND } r2$
0010	MOVE	Move entre registradores	$r1 := r2$
0011	COMPL	Complemento	$r1 := \text{inv}(r2)$
0100	LSHIFT	Deslocamento à esquerda	$r1 := \text{lshift}(r2);$
0101	RSHIFT	Deslocamento a direita	$r1 := \text{rshift}(r2);$
0110	GETMBR	Armazena MBR em registrador	$r1 := \text{MBR}$
0111	TEST	Testa registrador	if $r2 < 0$ then $N = \text{true};$ if $r2 = 0$ then $Z = \text{true};$
1000	BEGRD	Início de leitura	$\text{mar} := r1; \text{rd};$
1001	BEGWR	Início de escrita	$\text{mar} := r1; \text{mbr} := r2; \text{wr};$
1010	CONRD	Continua leitura	$\text{rd};$
1011	CONWR	Continua escrita	$\text{wr};$
1100			
1101	NJUMP	Desvia se N	if N then goto addr
1110	ZJUMP	Desvia se Z	if Z then goto addr
1111	UJUMP	Desvio incondicional	goto addr

Formato da microinstrução:



0: <i>mar</i> := <i>pc</i> ; <i>rd</i> ;	28: <i>tir</i> := <i>lshift</i> (<i>tir</i>) ;
1: <i>rd</i> ;	if <i>n</i> then goto 40;
<i>pc</i> := <i>pc</i> + 1 ;	29: <i>tir</i> := <i>lshift</i> (<i>tir</i>) ;
2: <i>ir</i> := <i>mbr</i> ;	if <i>n</i> then goto 35;
<i>tir</i> := <i>lshift</i> (<i>ir</i>) ; →	30: <i>alu</i> := <i>tir</i> ;
if <i>n</i> then goto 28;	if <i>n</i> then goto 33;
3: <i>tir</i> := <i>lshift</i> (<i>tir</i>) ;	
if <i>n</i> then goto 19;	31: <i>a</i> := <i>ir</i> ; {LODL}
4: <i>tir</i> := <i>lshift</i> (<i>tir</i>) ;	<i>a</i> := <i>a</i> + <i>sp</i> ;
if <i>n</i> then goto 11;	32: <i>mar</i> := <i>a</i> ; <i>rd</i> ;
5: <i>alu</i> := <i>tir</i> ;	<i>rd</i> ;
if <i>n</i> then goto 09;	<i>ac</i> := <i>mbr</i> ;
	goto 0;
6: <i>mar</i> := <i>ir</i> ; <i>rd</i> ; {LODD}	
7: <i>rd</i> ;	33: <i>a</i> := <i>ir</i> ; {STOL}
8: <i>ac</i> := <i>mbr</i> ;	<i>a</i> := <i>a</i> + <i>sp</i> ;
goto 0;	34: <i>mar</i> := <i>a</i> ; <i>mbr</i> := <i>ac</i> ; <i>wr</i> ;
9: <i>mar</i> := <i>ir</i> ; <i>mbr</i> := <i>ac</i> ; <i>wr</i> ; {STOD}	<i>wr</i> ;
10: <i>wr</i> ;	goto 0;
goto 0;	
11: <i>alu</i> := <i>tir</i> ;	35: <i>alu</i> := <i>tir</i> ;
if <i>n</i> then goto 15;	if <i>n</i> then goto 38;
12: <i>mar</i> := <i>ir</i> ; <i>rd</i> ; {ADDD}	
13: <i>rd</i> ;	36: <i>a</i> := <i>ir</i> ; {ADDL}
14: <i>a</i> := <i>mbr</i> ;	<i>a</i> := <i>a</i> + <i>sp</i> ;
<i>ac</i> := <i>ac</i> + <i>a</i> ;	37: <i>mar</i> := <i>a</i> ; <i>rd</i> ;
goto 0;	<i>rd</i> ;
	<i>a</i> := <i>mbr</i> ;
15: <i>mar</i> := <i>ir</i> ; <i>rd</i> ; {SUBD}	<i>ac</i> := <i>ac</i> + <i>a</i> ;
16: <i>rd</i> ;	goto 0;
99: <i>ac</i> := <i>ac</i> + 1 ;	
17: <i>a</i> := <i>mbr</i> ;	38: <i>a</i> := <i>ir</i> ; {SUBL}
<i>a</i> := <i>inv</i> (<i>a</i>) ;	<i>a</i> := <i>a</i> + <i>sp</i> ;
18: <i>ac</i> := <i>ac</i> + <i>a</i> ;	39: <i>mar</i> := <i>a</i> ; <i>rd</i> ;
goto 0;	<i>rd</i> ;
	goto 99;


```

12: mar := ir ; rd ; {ADDD}
13: rd ;
14: a := mbr ;
   ac := ac + a ;
   goto 0;
15: mar := ir ; rd ; {SUBD}
16: rd ;
99: ac := ac + 1;
17: a := mbr ;
   a := inv(a) ;
18: ac := ac + a ;
   goto 0;
19: tir := lshift(tir) ;
   if n then goto 25;
20: alu := tir ;
   if n then goto 23;
21: alu := ac ; {JPOS}
   if n then goto 0;
22: pc := ir ;
   pc := band(pc , amask) ;
   goto 0;
23: alu := ac ; {JZER}
   if z then goto 22;
24: goto 0;
25: alu := tir ;
   if n then goto 27;
26: pc := ir ; {JUMP}
   pc := band(pc , amask) ;
   goto 0;
27: ac := ir ; {LOCO}
   ac := band(ac , amask) ;
   goto 0;

```

```

36: a := ir ; {ADDL}
   a := a + sp ;
37: mar := a ; rd ;
   rd ;
   a := mbr ;
   ac := ac + a ;
   goto 0;
38: a := ir ; {SUBL}
   a := a + sp ;
39: mar := a ; rd ;
   rd ;
   goto 99;
40: tir := lshift(tir) ;
   if n then goto 46;
41: alu := tir ;
   if n then goto 44;
42: alu := ac ; {JNEG}
   if n then goto 22;
43: goto 0;
44: alu := ac ; {JNZE}
   if z then goto 0;
45: pc := ir ;
   pc := band(pc , amask) ;
   goto 0;
46: tir := lshift(tir) ;
   if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp ; mbr := pc ; wr ;
   wr ;
49: pc := ir ;
   pc := band(pc , amask) ;
   goto 0;

```

```

50: tir := lshift (tir);
    if n then goto 65;
51: tir := lshift (tir);
    if n then goto 59;
52: alu := tir;
    if n then goto 56;

53: mar := ac; rd; {PSHI}
    rd;
54: sp := sp + (-1);
55: a := mbr;
    mar := sp; mbr := a; wr;
    wr;
    goto 0;

56: mar := sp; rd; {POPI}
57: rd;
    sp := sp + 1;

58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
59: alu := tir;
    if n then goto 62;

60: sp := sp + (-1); {PUSH}
61: mar := sp; mbr := ac; wr;
    wr;
    goto 0;

62: mar := sp; rd; {POP}
63: rd;
    sp := sp + 1;

```

```

64: ac := mbr;
    goto 0;

65: tir := lshift (tir);
    if n then goto 73;
66: alu := tir;
    if n then goto 70;

67: mar := sp; rd; {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;

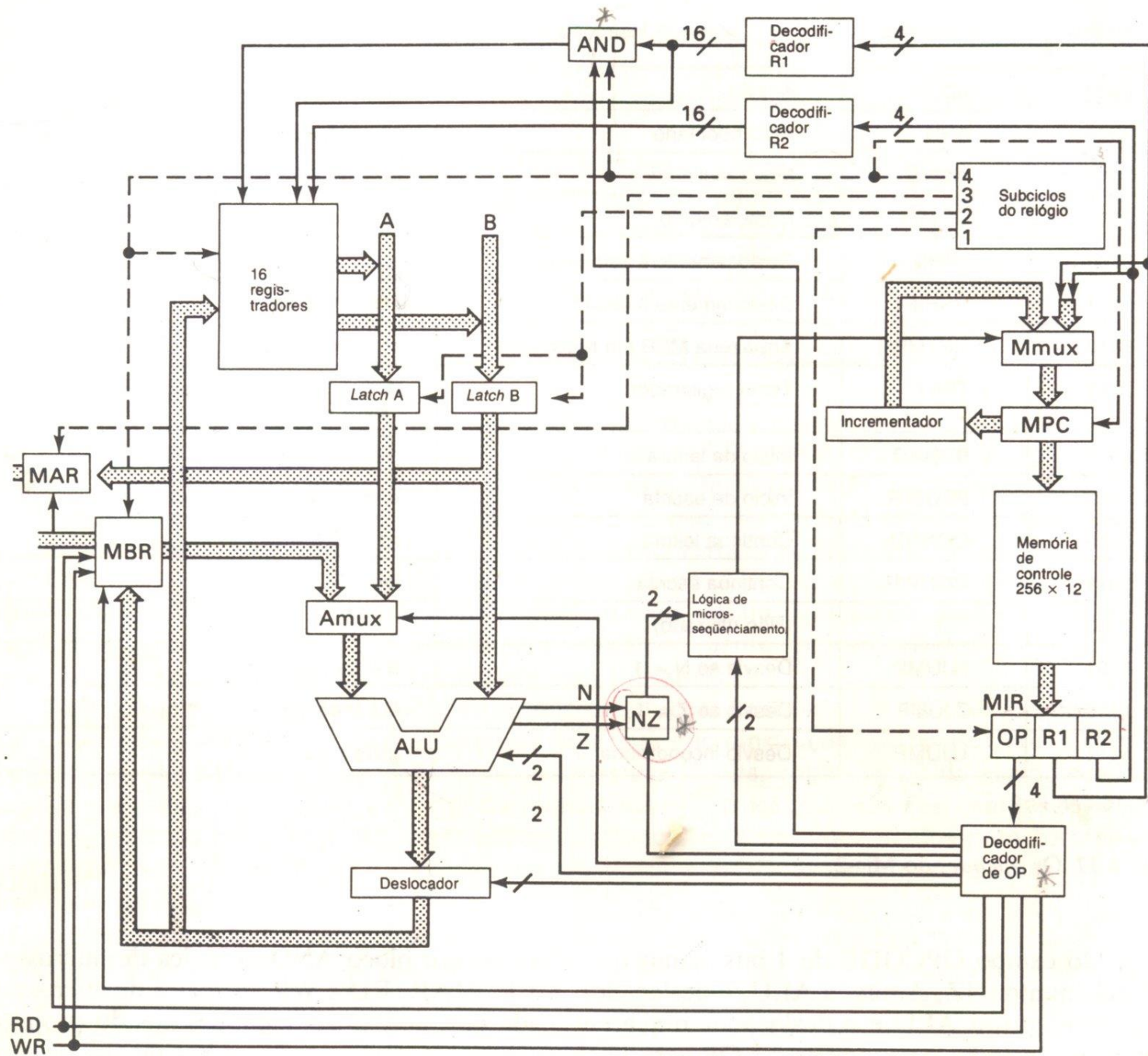
70: a := ac; {SWAP}
71: ac := sp;
72: sp := a;
    goto 0;

73: alu := tir;
    if n then goto 76;

74: a := ir; {INSP}
    a := band (a, smask);
75: sp := sp + a;
    goto 0;

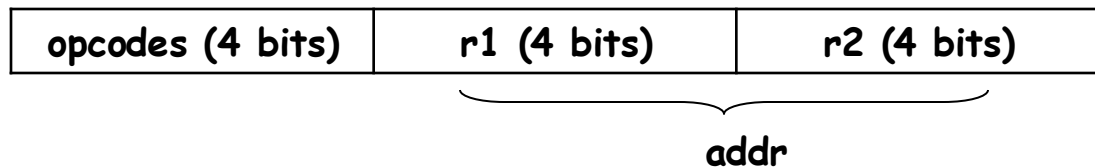
76: a := ir; {DESP}
    a := band (a, smask);
77: a := inv (a);
78: a := a + 1;
    sp := sp + a;
    goto 0;

```



Binário	Mnemônico	Instrução	Significado
0000	ADD	Adição	$r1 := r1 + r2$
0001	AND	E booleano	$r1 := r1 \text{ AND } r2$
0010	MOVE	Move entre registradores	$r1 := r2$
0011	COMPL	Complemento	$r1 := \text{inv}(r2)$
0100	LSHIFT	Deslocamento à esquerda	$r1 := \text{lshift}(r2);$
0101	RSHIFT	Deslocamento a direita	$r1 := \text{rshift}(r2);$
0110	GETMBR	Armazena MBR em registrador	$r1 := \text{MBR}$
0111	TEST	Testa registrador	if $r2 < 0$ then $N = \text{true};$ if $r2 = 0$ then $Z = \text{true};$
1000	BEGRD	Início de leitura	$\text{mar} := r1; \text{rd};$
1001	BEGWR	Início de escrita	$\text{mar} := r1; \text{mbr} := r2; \text{wr};$
1010	CONRD	Continua leitura	$\text{rd};$
1011	CONWR	Continua escrita	$\text{wr};$
1100			
1101	NJUMP	Desvia se N	if N then goto addr
1110	ZJUMP	Desvia se Z	if Z then goto addr
1111	UJUMP	Desvio incondicional	goto addr

Formato da microinstrução:



opcodes		ALU ^H	ALU ^L	SH ^H	SH ^L	NZ	AMUX	AND	MAR	MBR	RD	WR	MSL ^H	MSL ^L
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+		+		+						
5	RSHIFT	+			+	+		+						
6	GETMBR	+				+	+	+						
7	TEST	+				+								
8	BEGRD	+							+		+			
9	BEGWR	+							+	+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+

+ → bit ativado

H → bit mais a esquerda (high)

L → bit mais a direita (low)

MSL → equivalente aos bits COND

Usando ainda menos memória

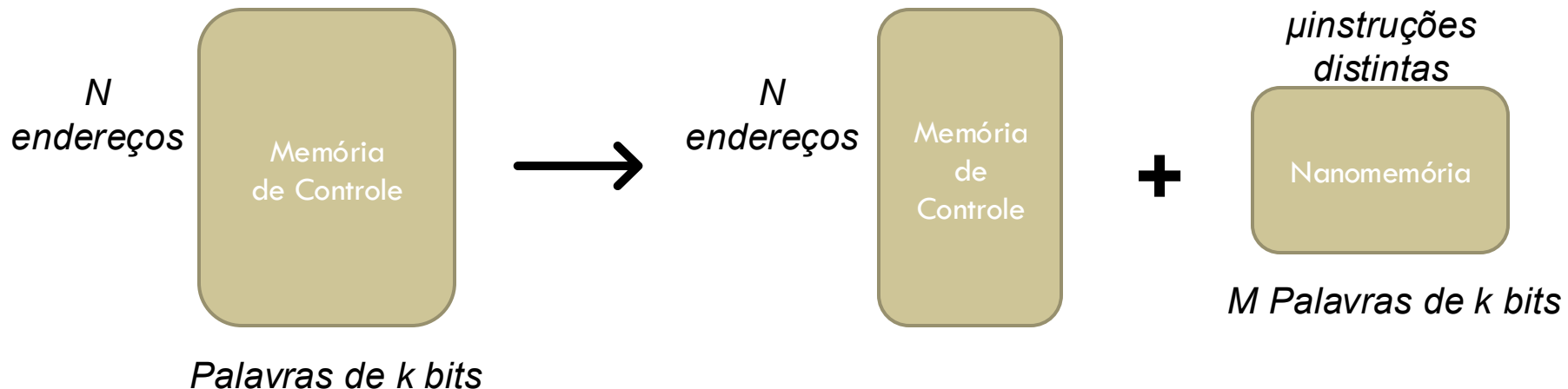
- Que é um programa? Para que serve memória?
- Tem alguma diferença entre a Memória de Controle e a Memória Principal?

Usando ainda menos memória

- Que é um programa? Para que serve memória?
- **Nanoprogramação** pode útil quando temos muitas (micro)instruções repetidas na (micro)programa

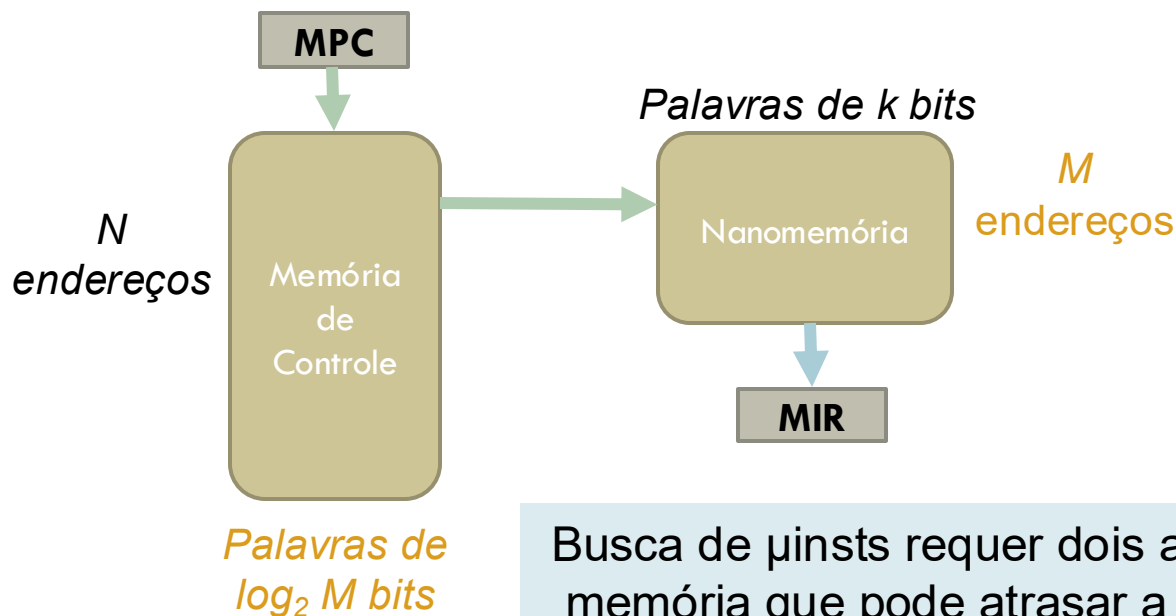


- Utiliza uma memória adicional (dentro a UC), para guardar μ instruções distintas, chamada uma nanomemória



Nanoprogramação e Nanomemória

- Um programa é uma sequência de instruções
 - Se a nanomemória guarda as M μinstruções (distintas), o que temos na Memória de Controle?
 - A sequência, na forma de ponteiros (endereços da nanomemória)

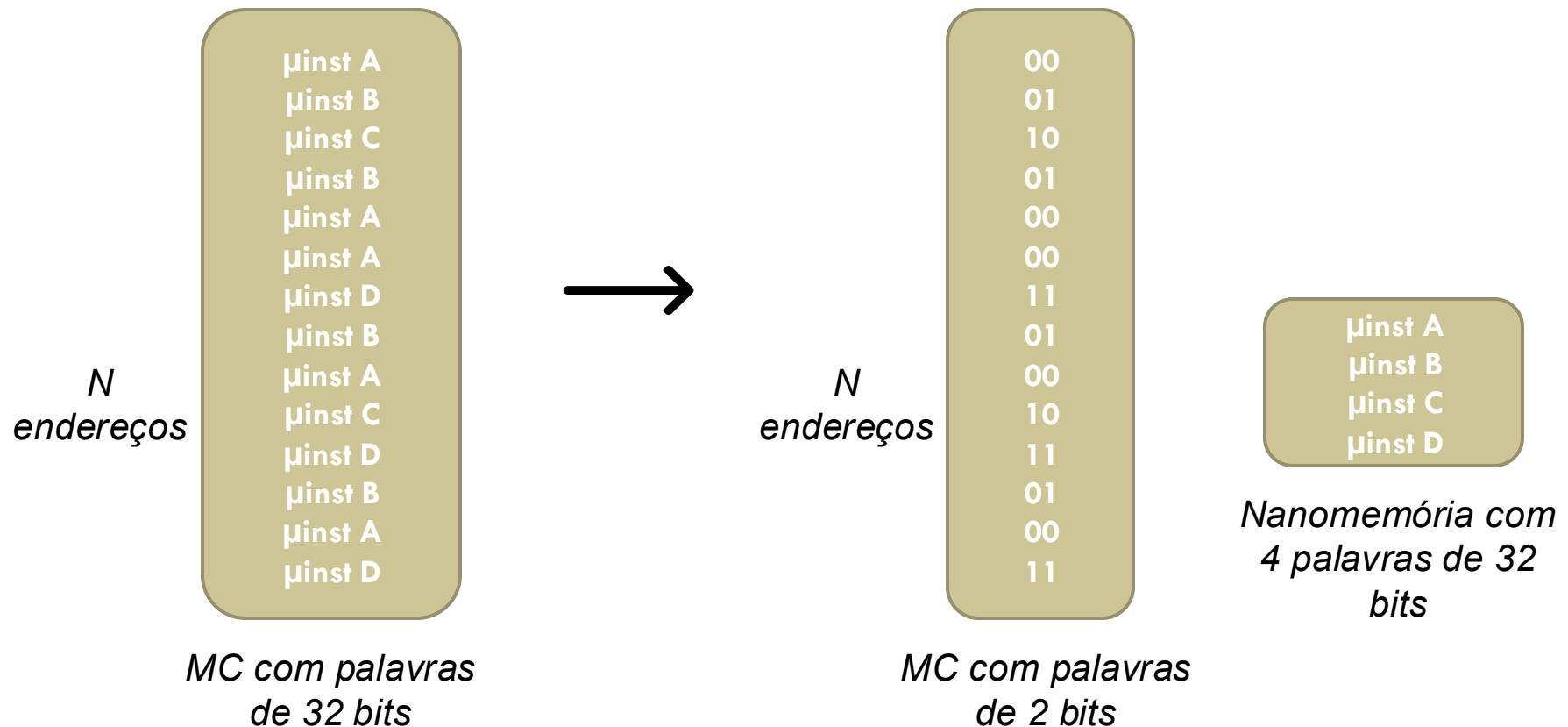


Se N é muito maior de M
então $N.k > N.\log_2 M + M.k$
Isto é, a UC seria mais barata
por requer menos espaço

Busca de μinsts requer dois acessos de memória que pode atrasar a execução

Exemplo de Nanoprogramação

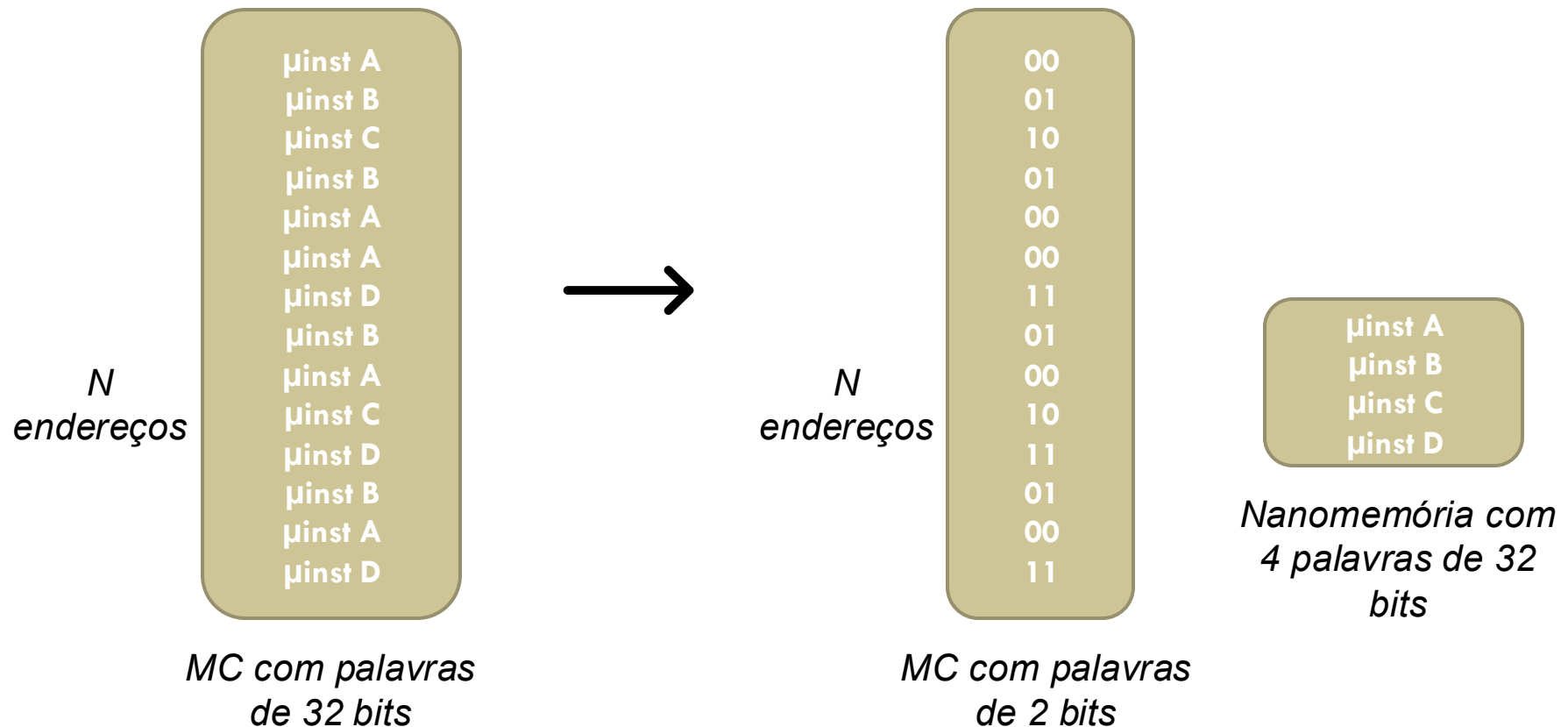
- A Nanomemória guarda as instruções distintas



Em termos de espaço, dado $M = 4$, vale a pena para programas de que tamanho?

Exemplo de Nanoprogramação

- A Nanomemória guarda as instruções distintas



Em termos de espaço, dado $M = 4$, vale a pena para programas de que tamanho?

$$32 \times N > 2 \times N + 128 \Rightarrow N > 128/30$$

O que vem a seguir em Arq. Comp.?

- Processadores executam programas sequenciais seguindo o modelo Von Neumann
 - A gente projetou um em poucas semanas...
 - O que, então, a comunidade/indústria tem feito nos últimos 70 anos?

O que vem a seguir em Arq. Comp.?

- Processadores executam programas sequenciais seguindo o modelo Von Neumann
 - A gente projetou um em poucas semanas...
 - O que, então, a comunidade/indústria tem feito nos últimos 70 anos?
 - Faça tudo em hardware?
 - Tudo = O Modelo Von Neumann
 - Acaba com a vantagem de Opção B
 - **Optimizações e melhorias**
 - Para uma execução ainda mais *eficiente*
 - RISC e Arquiteturas Superescalares com micro-operações



O que vem a seguir em Arq. Comp.?

- Processadores executam programas sequenciais seguindo o modelo Von Neumann
 - A gente projetou um em poucas semanas...
 - O que, então, a comunidade/indústria tem feito nos últimos 70 anos?
 - **Que mais temos que aprender?**
 - **Que é um computador?**
 - **Às vezes você tem que dar um passo para trás para dar dois para frente ...**

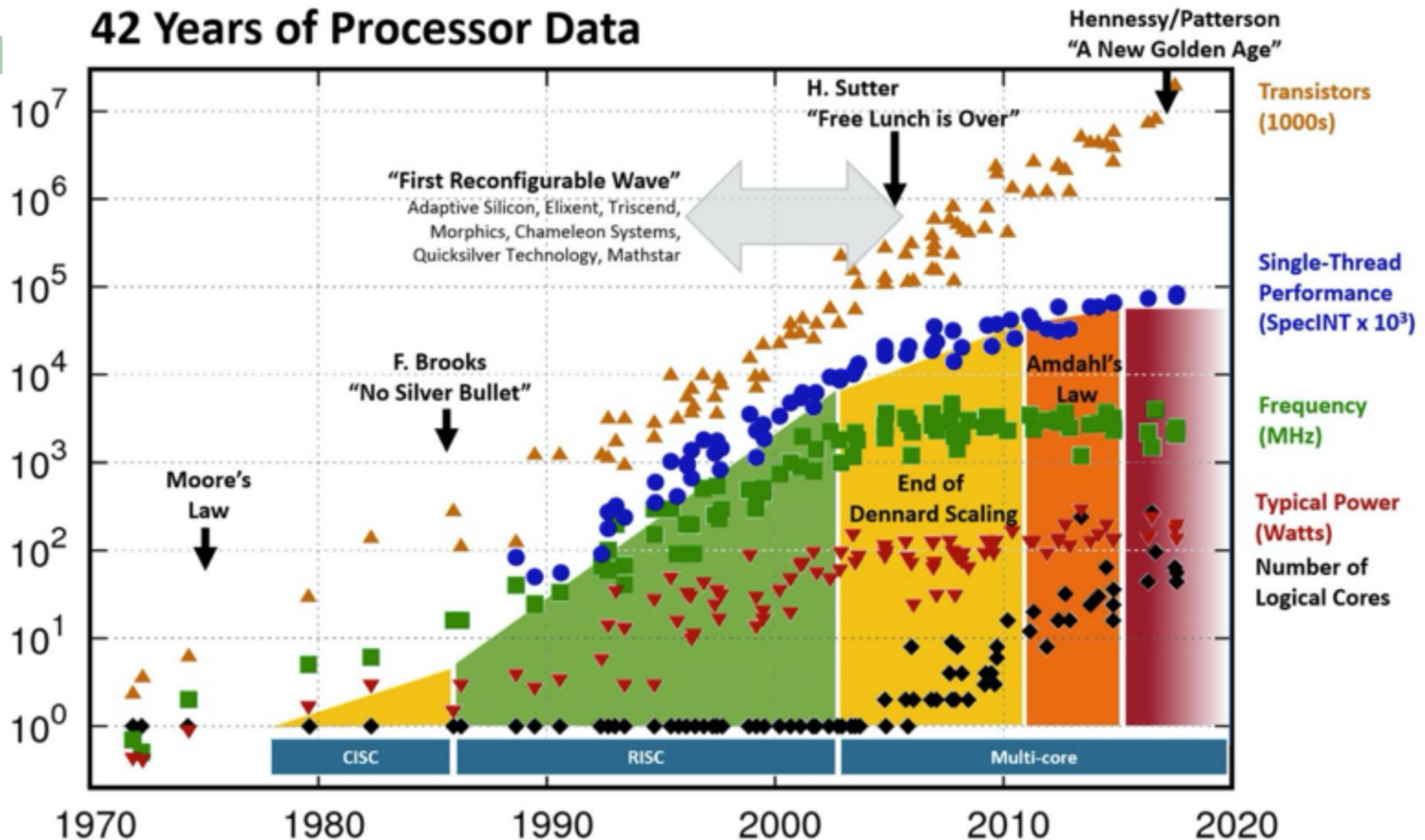
O que vem a seguir em Arq. Comp.?

- Processadores executam programas sequenciais seguindo o modelo Von Neumann
 - O que, então, a comunidade/indústria tem feito nos últimos 70 anos?
 - Que mais temos que aprender?
 - E a Lei de Moore (2x transistores cada 2 anos) e *Dennard's Scaling* (2x desempenho/watt cada 18 meses)?
 - O consumo de energia dinâmica dos circuitos CMOS é proporcional à frequência.
 - Historicamente, a redução de potência do transistor proporcionada pelo Scaling de Dennard permitiu que os fabricantes aumentassem as frequências do relógio de uma geração para a seguinte, sem aumentar significativamente o consumo de energia total do circuito.

O que vem a seguir em Arq. Comp.?

- Processadores executam programas sequenciais seguindo o modelo Von Neumann
 - O que, então, a comunidade/indústria tem feito nos últimos 70 anos?
 - Que mais temos que aprender?
 - E a Lei de Moore (2x transistores cada 2 anos) e *Dennard's Scaling* (2x desempenho/watt cada 18 meses)?
 - O consumo de energia dinâmica dos circuitos CMOS é proporcional à frequência.
 - Historicamente, a redução de potência do transistor proporcionada pelo Scaling de Dennard permitiu que os fabricantes aumentassem as frequências do relógio de uma geração para a seguinte, sem aumentar significativamente o consumo de energia total do circuito.
 - Mas além disso, podemos otimizar "o modelo de Von Neumann"?

O que vem a seguir? Quem sabe?



Hennessey and Patterson, Turing Lecture 2018, overlaid over "42 Years of Processors Data"

<https://www.karlsruhp.net/2018/02/42-years-of-microprocessor-trend-data/>; "First Wave" added by Les Wilson, Frank Schirrmeister

Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten

New plot and data collected for 2010-2017 by K. Rupp

US\$1000 de poder computacional



O que vem a seguir? Quem sabe?

- Duas coisas aconteceram na mesma época em meados dos anos 2000.
- **Primeiro**, *Dennard's Scaling* realmente chegou ao fim.
 - *Dennard's Scaling* permitiu que um processo de fabricação pode escalar para um outro e aumentar o desempenho sem um aumento significativo do consumo de energia.
 - Antes, as taxas de relógio aumentaram, chegando a cerca de 3-4 GHz, mas isso acabou.
 - O CTO da Intel apontou que se continua assim, as futuras densidades de energia se aproximaram daquelas do núcleo de reatores nucleares ou do centro do sol.
 - A partir desse ponto, em vez de aumentar as taxas de relógio, o poder de computação adicional foi fornecido na forma de mais núcleos.

O que vem a seguir? Quem sabe?

- Duas coisas aconteceram na mesma época em meados dos anos 2000.
- A **segunda** coisa foi que os arquitetos tinham feito tudo o que podiam pensar para melhorar o desempenho de um processador de uso geral: pipelines, caches multinível, execução fora de ordem, previsão de desvios, multicore, ...
 - Se você (o programador/usuário) precisa obter mais desempenho, não o obterá de graça apenas esperando, como aconteceu naquele período anterior.
 - Você pode ter mais de tudo... mais núcleos de uso geral, mais processadores de uso geral.
 - O que você não pode ter é mais desempenho para uma única tarefa executada em um processador de uso geral.

A Nova Era Dourada de Arq. Comp.

- Se você não consegue obter mais desempenho com um processador de uso geral, a resposta é usar processadores de uso específico.
- Uma idade de ouro, por quê? Arquiteturas novas e inovadoras serão importantes e não serão superadas apenas alguns anos depois pelas melhorias na computação de uso geral.
- Estamos agora na era da computação de domínios específicos.
 - Se olharmos para o quadro geral, um processador de uso geral começa a parecer ainda menos a solução certa em todas as circunstâncias.
 - Muitos algoritmos são inerentemente paralelos. Na maioria das linguagens de programação, como C++ ou Python, o programador precisa ordenar essas operações em um fluxo sequencial.

O que vem a seguir para a gente?

- Processadores executam programas sequências seguindo o modelo Von Neumann
 - A gente projetou um processador em poucas semanas...
 - O que, então, a comunidade/indústria tem feito nos últimos 70 anos?
 - Que mais temos que aprender?
- Como criar um processador que é mais rápido sem ser mais caro e que continua ser fácil de usar?
 - A facilidade vem de abstração e a realização de traduções de programas escrita em linguagens de programação (sequenciais) de alto nível
 - Por serem linguagens sequenciais, o fator limitante é que os processadores são obrigados a seguir o modelo Von Neumann (A Lei de Programação) ...
 - Como acelerar algo que é inerentemente sequencial?
- Bem-vindo ao mundo da **Microarquitetura!** (*hardware x software*)

O que vem a seguir para a gente?

- Processadores executam programas sequências seguindo o modelo Von Neumann
 - A gente projetou um processador em poucas semanas...
 - O que, então, a comunidade/indústria tem feito nos últimos 70 anos?
 - Que mais temos que aprender?
- Como criar um processador que é mais rápido sem ser mais caro e que continua ser fácil de usar?
 - A facilidade vem de abstração e a realização de traduções de programas escrita em linguagens de programação (sequenciais) de alto nível
 - Por serem linguagens sequenciais, o fator limitante é que os processadores são obrigados a seguir o modelo Von Neumann (A Lei de Programação) ...
 - Como acelerar algo que é inerentemente sequencial?
- Bem-vindo ao mundo da **Microarquitetura!**



But don't get seen!!

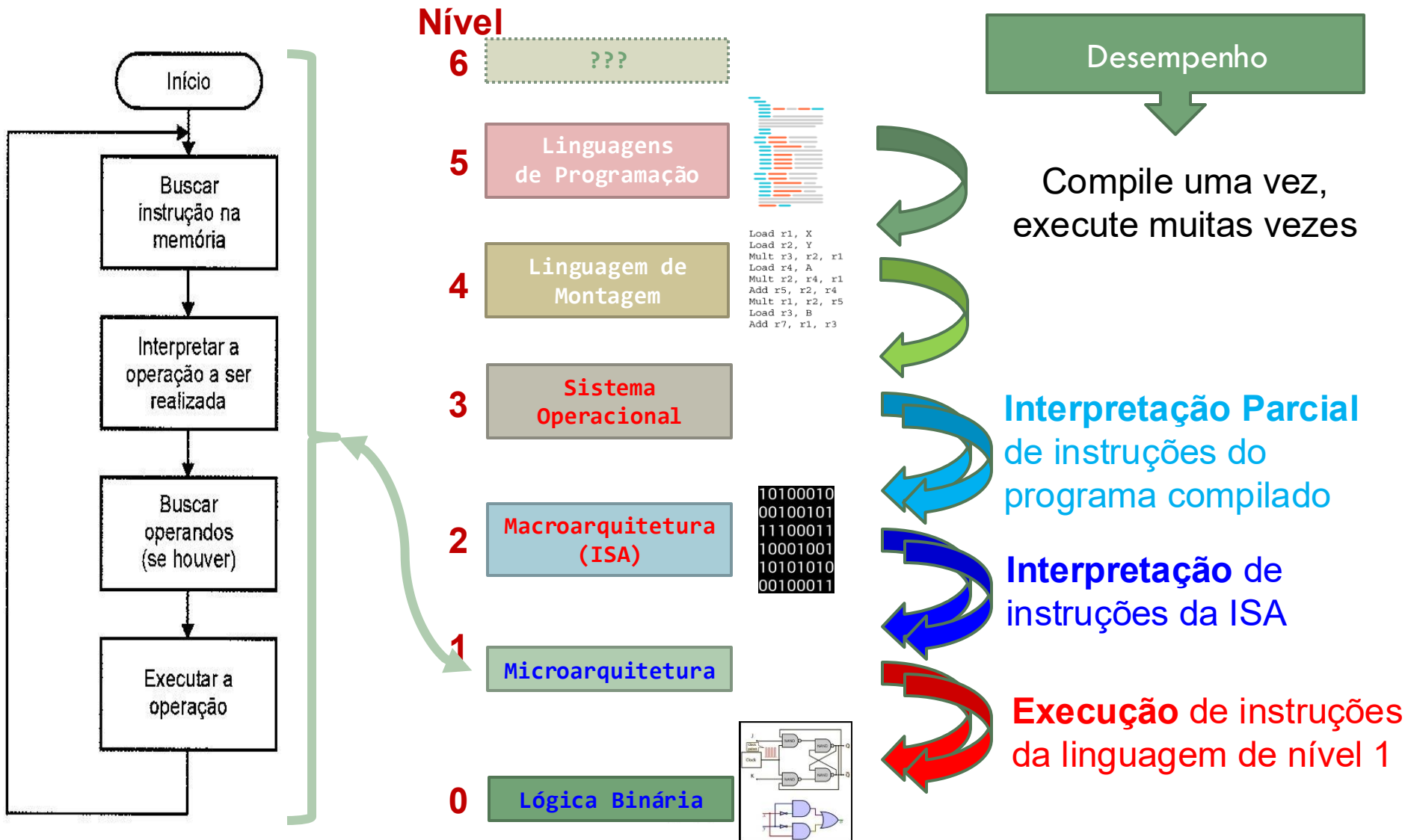
O que vem a seguir para a gente?

- Processadores executam programas sequências seguindo o modelo Von Neumann
 - A gente projetou um processador em poucas semanas...
 - O que, então, a comunidade/indústria tem feito nos últimos 70 anos?
 - Que mais temos que aprender?
- Como criar um processador que é mais rápido sem ser mais caro e que continua ser fácil de usar?
 - A facilidade vem de abstração e a realização de traduções de programas escrita em linguagens de programação (sequenciais) de alto nível
 - Por ser linguagens sequenciais, o fator limitante é que processadores são obrigados a seguir o modelo Von Neumann (A Lei de Programação) ...
 - Como acelerar algo que é inerentemente sequencial?
- Bem-vindo ao mundo da **Microarquitetura!**



Because you don't have to break the law to break or bend the rules!!

O que acontece na arquitetura?



Desempenho arquitetural

- O tempo de execução depende nas características do sistema todo, mas onde podemos achar melhorias?

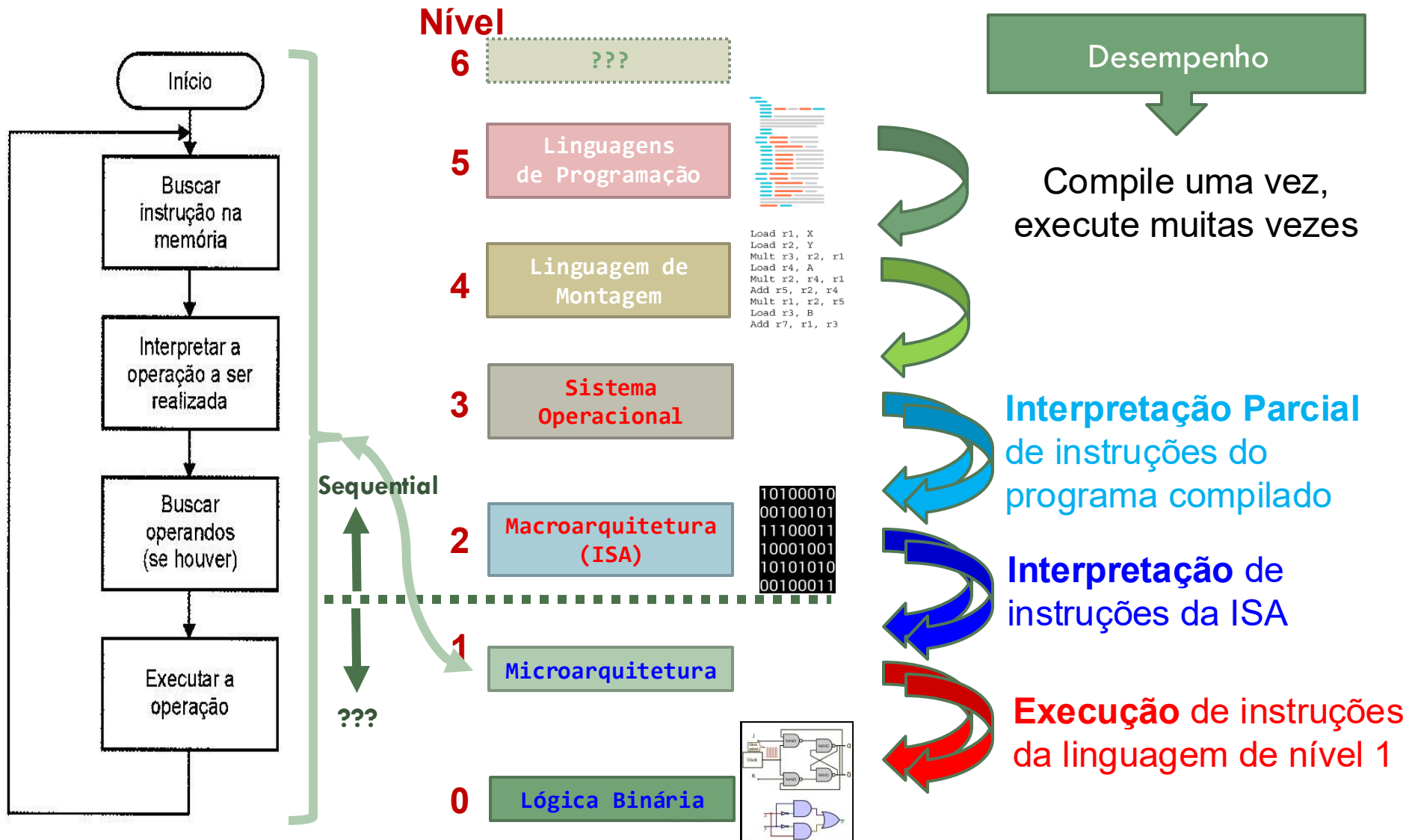
The diagram illustrates the equation for execution time: $\frac{\text{Tempo}}{\text{Programa}} = \frac{\text{Instruções}}{\text{Programa}} \times \frac{\text{Ciclos}}{\text{Instrução}} \times \frac{\text{Tempo}}{\text{Ciclo}}$. A blue curved arrow labeled 'Instruções executadas' points from the 'Instruções Programa' term to the 'Tempo Programa' term. The 'Instruções Programa' term is enclosed in a blue box, while the other two terms are in green boxes.

$$\frac{\text{Tempo}}{\text{Programa}} = \frac{\text{Instruções}}{\text{Programa}} \times \frac{\text{Ciclos}}{\text{Instrução}} \times \frac{\text{Tempo}}{\text{Ciclo}}$$

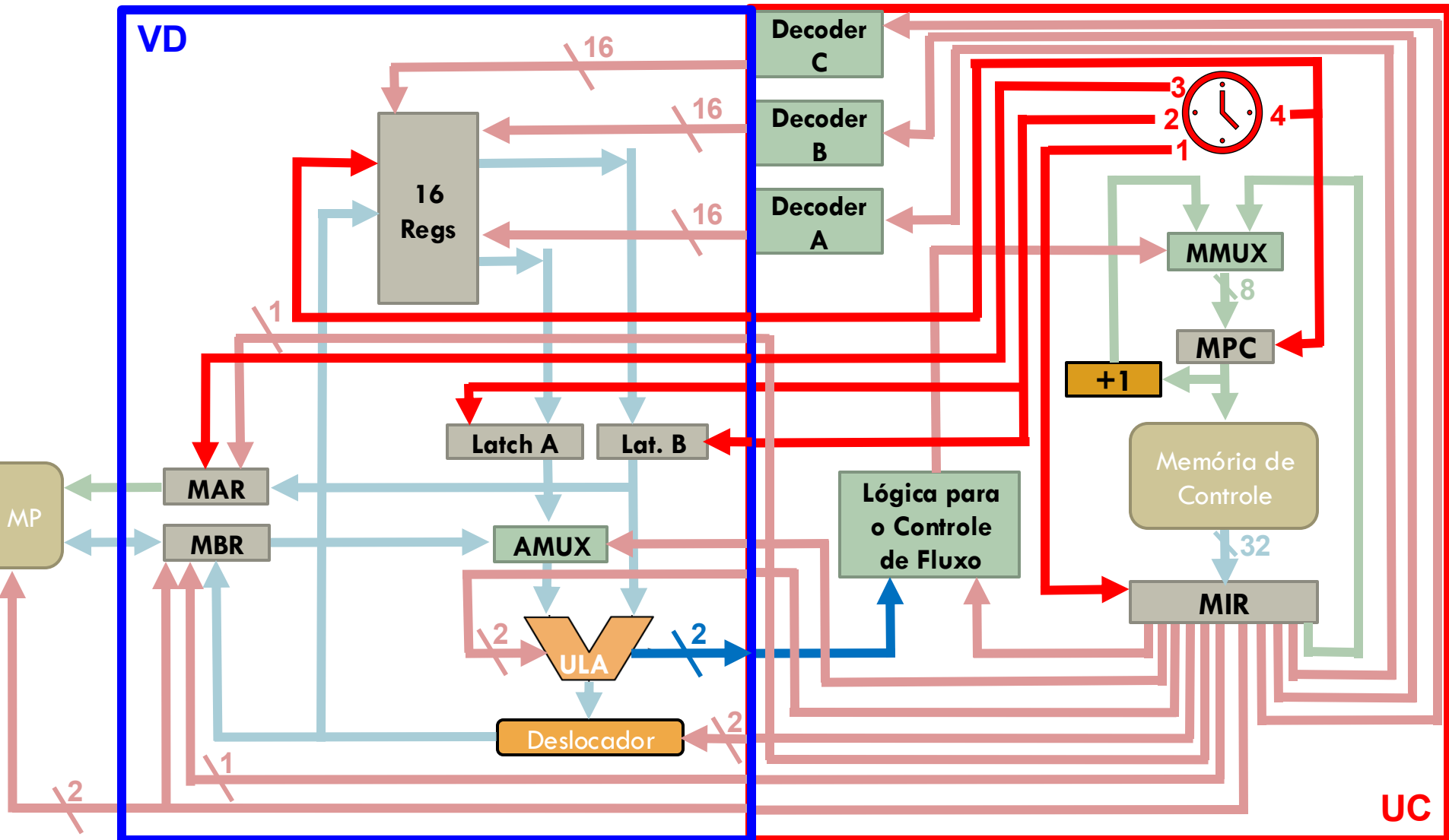
Depende no: *Compilador* *Microarquitetura* *Tecnologia*
Isto é na implementação adotada para cada nível

- Seria possível que algum desses elementos da equação pode chegar a ser menor que um?

Se não é sequencial, como fica?



O projeto do processador MIC-1



Intel Pentium 4 Northwood

Buffer Allocation & Register Rename

Execution Pipeline Start

Instruction Trace Cache

Trace Cache Access, next Address Predict

Instruction Queue (for less critical fields of the uOps)

General Instruction Address Queue & Memory Instruction Address Queue (queues register entries and latency fields of the uOps for scheduling)

Floating Point, MMX, SSE2 Renamed Register File 128 entries of 128 bit.

uOp Schedulers

FP Move Scheduler: (8x8 dependency matrix)

Parallel (Matrix) Scheduler for the two double pumped ALU's

General Floating Point and Slow Integer Scheduler: (8x8 dependency matrix)

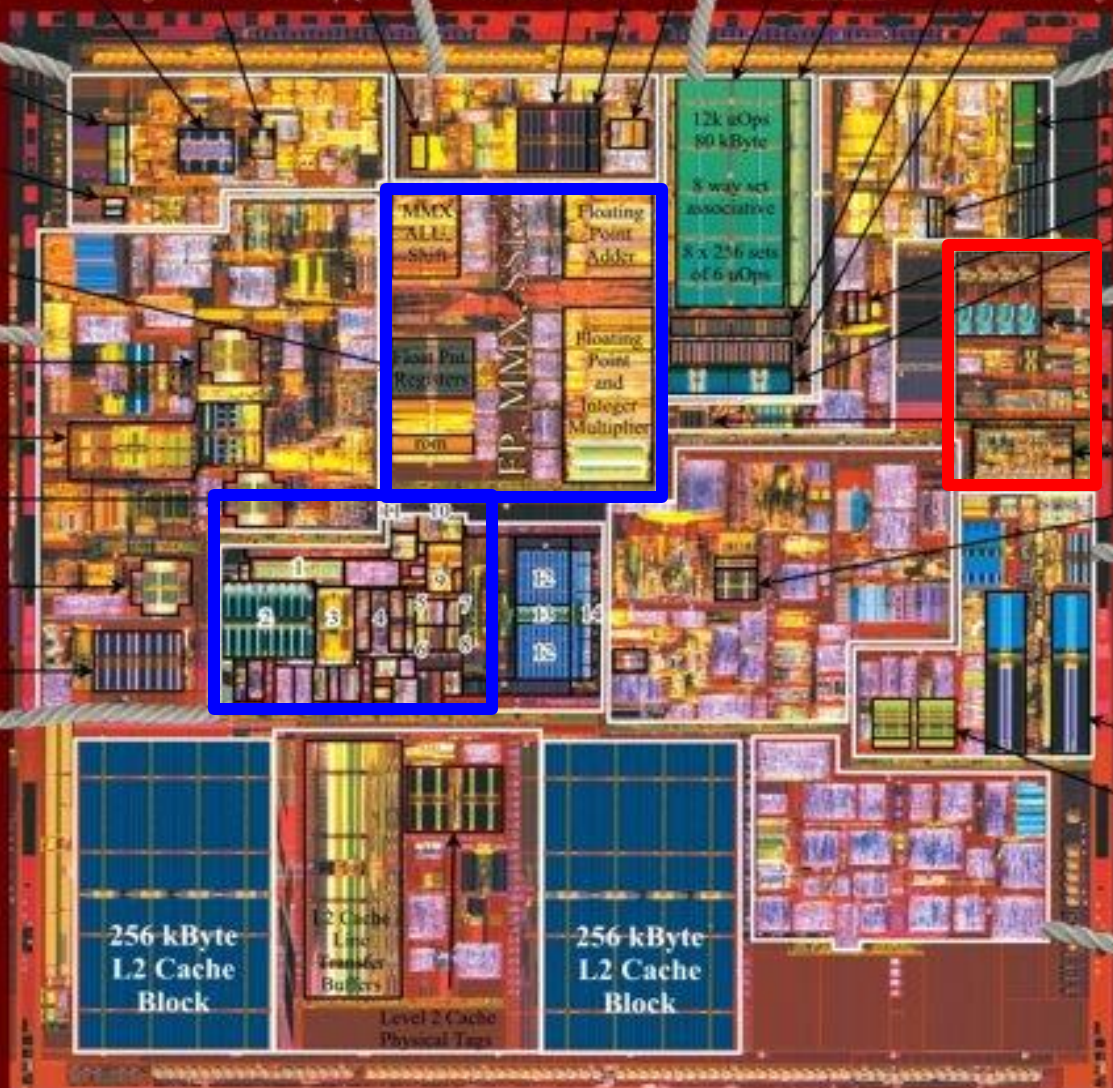
Load / Store uOp Scheduler: (8x8 dependency matrix)

Load / Store Linear Address Collision History Table

Integer Execution Core

- (1) uOp Dispatch unit & Replay Buffer Dispatches up to 6 uOps / cycle
- (2) Integer Renamed Register File 128 entries of 32 bit + 6 status flags 12 read ports and six write ports
- (3) Databus switch & Bypasses to and from the Integer Register File.
- (4) Flags, Write back
- (5) Double Pumped ALU 0
- (6) Double Pumped ALU 1
- (7) Load Address Generator Unit
- (8) Store Address Generator Unit
- (9) Load Buffer (48 entries)
- (10) Store Buffer (24 entries)

Register Alias History Tables (2x126)
Register Alias Tables uOp Queue
Micro code Sequencer
Micro code ROM & Flash
Trace Cache Fill Buffers
Distributed Tag comparators 24 bit virtual Tags



Trace Cache Branch Prediction Table (BTB), 512 entries.

Return Stacks (2x16 entries)

Trace Cache next IP's (2x)

Miscellaneous Tag Data

Instruction Decoder

Up to 4 decoded uOps/cycle out (from max. one x86 instr/cycle) Instructions with more than four are handled by Micro Sequencer

Trace Cache LRU bits
Raw Instruction Bytes in Data TLB, 64 entry fully associative, between threads dual ported (for loads and stores)

Instruction Fetch from L2 cache and Branch Prediction

Front End Branch Prediction Tables (BTB), shared, 4096 entries in total

Instruction TLB's 2x64 entry, fully associative for 4k and 4M pages. In: Virtual address [31:12] Out: Physical address [35:12] + 2 page level bits

Front Side Bus Interface, 400..800 MHz

- (11) ROB Reorder Buffer 3x42 entries
- (12) 8 kByte Level 1 Data cache four way set associative, 1R/1W

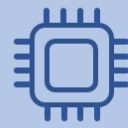
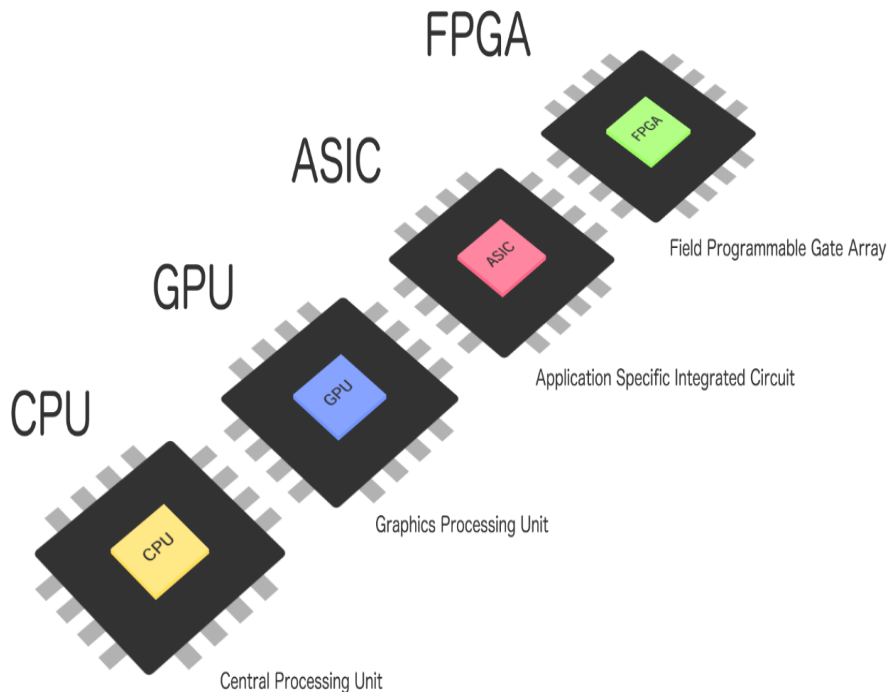
- (13) Summed Address Index decode and Way Predict
- (14) Cache Line Read / Write Transfer buffers and 256 bit wide bus to and from L2 cache

A Nova Era Dourada de Arq. Comp.

- O compilador do código-fonte toma decisões realmente detalhadas sobre exatamente quais instruções, em que ordem, devem operar em quais registradores.
- Mas o processador então pega esta código, altamente otimizado para execução sequencial, e tenta encontrar paralelismo.
 - Ele substitui as decisões tomadas pelo compilador quanto à ordem das instruções, reordenando-as, e os registradores a serem usados, renomeando-os.
 - Ele adivinha quais acessos à memória serão necessários, usando caches, que funcionam na maioria das vezes.
 - Ele adivinha para que lado os desvios irão, possivelmente executando instruções erradas.
 - E então, talvez a parte mais incrível de todas, ele faz tudo isso de uma maneira que todas as instruções parecem ser executadas na ordem exata em que o programador e o compilador escreveram em primeiro lugar.
- Mas para algoritmos especializados, com paralelismo e algoritmos bem compreendidos, jogar todo o conhecimento fora e tentar recuperá-lo rapidamente pode não ser o ideal abordagem.

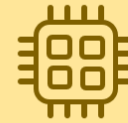
Que pode ser o futuro de Arq. Comp?

- Já temos CPUs com GPUs, FPGAs e N/TPUs juntos.
 - Por ex. Intel's 7nm Meteor Lake Chiplet



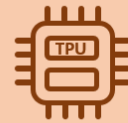
CPU

- Small models
- Small datasets
- Useful for design space exploration



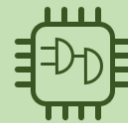
GPU

- Medium-to-large models, datasets
- Image, video processing
- Application on CUDA or OpenCL



TPU

- Matrix computations
- Dense vector processing
- No custom TensorFlow operations

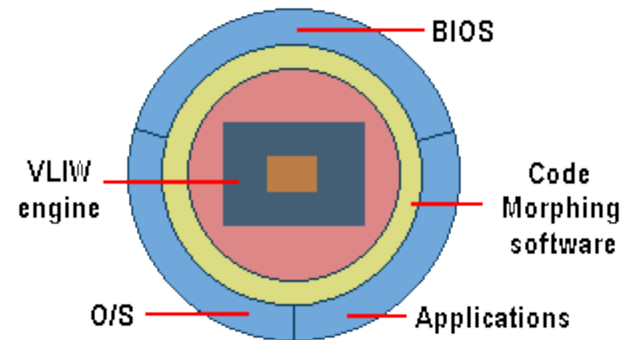


FPGA

- Large datasets, models
- Compute intensive applications
- High performance, high perf./cost ratio

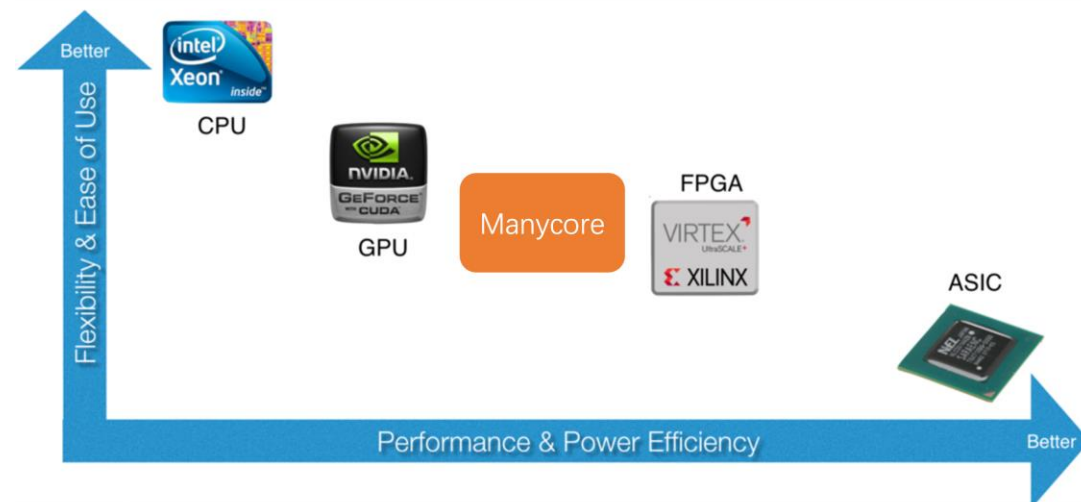
Que pode ser o futuro de Arq. Comp?

- O que seria possível se a microarquitetura pudesse executar qualquer microinstrução que desejasse?



Transmeta Crusoe Processor (1996-2009)

- Por que não usar hardware programável para redesenhar a microarquitetura em tempo real?



Que pode ser o futuro de Arq. Comp?

❖ What is Code Morphing?

Code morphing is basically binary translation so that the implementation of the ISA can be different from the software level (external ISA presented to users).

This approach allows a simple, compact, low power microprocessor implementation, with the freedom to modify the internal ISA between generations, while supporting the broad range of legacy x86 software available.



❖ What is Transmeta Crusoe?

VLIW processor and Code Morphing System (CMS) present an approach unique among commercial architectures: a microprocessor system with an internal VLIW instruction set architecture (ISA) with little resemblance to external ISA x86. The instruction is called a molecule and consists of 2 or 4 atoms (RISC like instructions).

Que pode ser o futuro de Arq. Comp?

- O que seria possível se a microarquitetura pudesse executar qualquer microinstrução que desejasse?
- Por que não usar hardware programável para redesenhar a microarquitetura em tempo real?

